



Event Sistemi

Kullanımı	
pygame.event.get() o karedeki tüm olayları liste olarak döndürür. Her karede çağrılmalı, aksi halde olay kuyruğu dolar.	
event.get()	Tüm olayları al ve temizle
event.peek()	Olayı kuyruğa bırakarak kontrol et
event.clear()	Tüm olayları temizle

Temel Olay İşleme Şablonu	
<pre>for olay in pygame.event.get(): if olay.type == pygame.QUIT: çalıştır = False elif olay.type == pygame.KEYDOWN: if olay.key == pygame.K_ESCAPE: çalıştır = False elif olay.type == pygame.MOUSEBUTTONDOWN: if olay.button == 1: # sol tık print("Tiklandi:", olay.pos)</pre>	

Olay Türleri	Açıklama
QUIT	Pencere kapatma düğmesine basıldı
KEYDOWN	Tuşa basıldı (bir kez)
KEYUP	Tuş bırakıldı
MOUSEBUTTONDOWN	Fare düğmesine basıldı
MOUSEBUTTONUP	Fare düğmesi bırakıldı
MOUSEMOTION	Fare hareket etti

Klavye Girdileri

KEYDOWN vs	Farkı
Yöntem	Kullanım Amacı
KEYDOWN	Tek seferlik eylem (ateş et, zıpla)
get_pressed()	Sürekli hareket (karakter kaydırma)

KEYDOWN ile Tuş Yakalama	
<pre>for olay in pygame.event.get(): if olay.type == pygame.KEYDOWN: if olay.key == pygame.K_SPACE: mermi_at() # bir kez ates elif olay.key == pygame.K_RETURN: oyunu_baslat()</pre>	

Key.get_pressed() ile Sürekli Hareket	
<pre>tuslar = pygame.key.get_pressed() if tuslar[pygame.K_LEFT]: x -= hiz if tuslar[pygame.K_RIGHT]: x += hiz if tuslar[pygame.K_UP]: y -= hiz if tuslar[pygame.K_DOWN]: y += hiz</pre>	

Sık Kullanılan Tuş Sabitleri	
Sabit	Tuş
K_UP / K_DOWN	Yukarı / Aşağı ok
K_LEFT / K_RIGHT	Sol / Sağ ok
K_SPACE	Boşluk çubuğu
K_ESCAPE	Esc tuşu
K_RETURN	Enter tuşu
K_a ... K_z	A'dan Z'ye harfler
K_0 ... K_9	0'dan 9'a sayılar
K_F1 ... K_F12	Fonksiyon tuşları

Fare Girdileri

Fare Olayları	
Olay / Özellik	Açıklama
MOUSEBUTTONDOWN	Düğmeye basıldı
MOUSEBUTTONUP	Düğme bırakıldı
MOUSEMOTION	Fare hareket etti
olay.pos	Tıklama koordinatı (x, y)
olay.button	Hangi düğme (1-5)
olay.rel	Göreceli hareket (dx, dy)

Fare Düğme Numaraları	
Numara	Düğme
1	Sol düğme
2	Orta düğme (tekerlek tıklama)
3	Sağ düğme
4	Tekerlek yukarı kaydırma
5	Tekerlek aşağı kaydırma

Fare Tıklama Yakalama	
<pre>for olay in pygame.event.get(): if olay.type == pygame.MOUSEBUTTONDOWN: fare_x, fare_y = olay.pos if olay.button == 1: # sol tık print("Sol:", fare_x, fare_y) elif olay.button == 3: # sag tık print("Sag:", fare_x, fare_y)</pre>	

Modülü	
mouse.get_pos()	Anlık fare konumu (x, y)
mouse.get_pressed()	Basılı düğmeler (sol, orta, sag)
mouse.set_visible()	İmleci gizle/göster
mouse.set_pos(x, y)	İmleci konumlandır

Hareket Mekaniği

Hız Formülü: Delta Time
Konum += Hız × dt
dt = son kare süresi (saniye cinsinden)
Delta time kullanmak oyunu FPS'ten bağımsız kılar.
60 FPS'de: dt ≈ 0.0167 sn → hiz * dt sabit.

```
Delta Time ile Hareket

saat = pygame.time.Clock()

x, y = 400.0, 300.0
vx, vy = 200.0, 0.0 # piksel/saniye

çalıştır = True
while çalıştır:
    dt = saat.tick(60) / 1000.0 # saniyeye çevir

    # Olaylari isle
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    # Hareketi güncelle
    x += vx * dt
    y += vy * dt

    # Ciz
    ekran.fill((0, 0, 0))
    pygame.draw.circle(ekran, (255,100,0),
        (int(x), int(y)), 20)
    pygame.display.flip()
```

Delta Time - Unutma!
Delta time **kullanmadan** hareket FPS'e bağlı olur:
60 FPS'de hız = 5 px/kare → saniyede 300 px
30 FPS'de hız = 5 px/kare → saniyede 150 px
dt ile: her FPS'de saniyede aynı piksel!

Sınır Kontrolü

3 Strateji	
Strateji	Davranış
Clamp	Sınırdan dur, geçemez
Wrap	Karşı kenardan çıkar
Bounce	Duvara çarpıp geri döner

Clamp: Sınırdan Dur
GENISLIK, YUKSEKLIK = 800, 600 boyut = 32 # x ekseninde sinirla x = max(0, min(x, GENISLIK - boyut)) # y ekseninde sinirla y = max(0, min(y, YUKSEKLIK - boyut))

Wrap: Karşıdan Çık
if x > GENISLIK: x = -boyut # sagdan cikinca soldan gir if x < -boyut: x = GENISLIK # soldan cikinca sagdan gir if y > YUKSEKLIK: y = -boyut if y < -boyut: y = YUKSEKLIK

Bounce: Geri Sek
if x <= 0 or x >= GENISLIK - boyut: vx = -vx # yatay hizi tersine ceviri x = max(0, min(x, GENISLIK - boyut)) if y <= 0 or y >= YUKSEKLIK - boyut: vy = -vy # dikey hizi tersine ceviri y = max(0, min(y, YUKSEKLIK - boyut))

Hızlı Kalıp: Hareket Eden Nesne

```
Tam Hareket Şablonu

import pygame

GENISLIK, YUKSEKLIK = 800, 600
HIZ = 250 # piksel/saniye

pygame.init()
ekran = pygame.display.set_mode(
    (GENISLIK, YUKSEKLIK))
saat = pygame.time.Clock()

x, y = GENISLIK // 2, YUKSEKLIK // 2
boyut = 32

çalıştır = True
while çalıştır:
    dt = saat.tick(60) / 1000.0

    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    # Klavye ile hareket
    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_LEFT]:
        x -= HIZ * dt
    if tuslar[pygame.K_RIGHT]:
        x += HIZ * dt
    if tuslar[pygame.K_UP]:
        y -= HIZ * dt
    if tuslar[pygame.K_DOWN]:
        y += HIZ * dt

    # Sınır kontrolu (Clamp)
    x = max(0, min(x, GENISLIK - boyut))
    y = max(0, min(y, YUKSEKLIK - boyut))

    # Ciz
    ekran.fill((20, 20, 40))
    pygame.draw.rect(ekran, (100, 200, 255),
        (int(x), int(y), boyut, boyut))
    pygame.display.flip()

pygame.quit()
```

Olay İşleme Komutları

Modülü	
Komut event.get() event.peek(t) event.set_blocked(t) event.set_allowed(t) event.post(olay) event.clear()	Açıklama Kuyruktaki tüm olayları al Türü kuyruğa dokunmadan kontrol et Belirli olay türünü engelle Engellenen türe izin ver Kuyruğa özel olay ekle Tüm kuyruğu temizle
Yılan Oyunu İpucu Yılanı bir liste olarak tut; her adımda yeni başı başa ekle , kuyruğu sondan sil : yilan.insert(0, yeni_bas) yilan.pop() Elma yenildiğinde pop() çağrılmaz → yılan uzar!	